Meilenstein 02 – AuthKit: Tests, Coverage und AuthenticationManager-Fix

Feld	Wert
Projekt	Overex
Zeitraum	2026-10-05 – 2026-10-07
Dokumentiert am	2026-10-10 20:28 CEST
Repository / Service	`authkit` (github.com/Umex10/authkit), Backend `apps/backend` (künftiger `auth-service` von Overex)
Thema	Testabdeckung des RS256-Backends: Unit-, Slice- und Integrationstests, JaCoCo, Fix einer Endlosrekursion im `AuthenticationManager`
Commits	8 (`92eb9b4` .. `119a0e9`), Branch `feature/rs256-jwks` , über PR #1 (`edd0e90`) nach `main` gemergt
Zeitaufwand	nicht erfasst
Bezug	Projektarbeit (5. Semester, Abgabe Jänner 2027) und Grundlage für die Bachelorarbeit (6. Semester)

Ziel

Nach Meilenstein 01 lief das Backend mit RS256, aber die Tests deckten den neuen Aufbau kaum ab. `MeControllerTest` scheiterte sogar, weil im Testkontext der öffentliche Key fehlte. Bevor AuthKit zum `auth-service` von Overex wird, sollte es ein verlässliches Sicherheitsnetz haben.

Ziel war deshalb:

- Die neuen Teile (Token-Claims, Token-Typ-Prüfung, JWKS, Rollen aus dem Token) mit Tests absichern.
- Messen, welcher Code überhaupt von Tests erreicht wird, und die Lücken schließen.
- Verstehen, wie Spring Security intern mit Providern und Managern arbeitet, um einen Fehler sauber zu beheben.
- Die Tests so ordnen, dass man Unit-, Slice- und Integrationstests getrennt starten kann.

Ausgangslage

- 24 Tests, davon 4 rot (`MeControllerTest` , fehlender öffentlicher Key im Testkontext). Stand aus Meilenstein 01.
- Keine Messung der Testabdeckung.
- Alle Tests lagen gemischt in den Paketen `auth` , `user` und `security` .
- `jwt.secret` und `JWT_SECRET` standen noch in Test-Properties, `docker-compose.yml` und der Doku, obwohl es HS256 nicht mehr gab.

Umsetzung (nach Commits)

Die Zeitpunkte der Commits stehen im Commit-Verlauf am Ende.

1. **`1c89034`** – Tests für die RS256-Claims (`sub` , `email` , `roles` ; Refresh-Token nur mit `sub`), ein `JwksControllerTest` (genau ein RSA-Key, kein privater Exponent `d`) und eine Test-Bean mit dem öffentlichen Key, damit `SecurityConfig` im `MeControllerTest` lädt.

2. **`e9f21bf`** – Beim Test „Refresh-Token als Access-Token“ trat ein `StackOverflowError` statt einer 401 auf. Gefixt, indem der `AuthenticationManager` als eigener `ProviderManager` mit `DaoAuthenticationProvider` gebaut wurde. Dazu `TestKeys` als gemeinsames Schlüsselpaar für alle Tests, weitere Negativtests im `JwtServiceTest` (falsche Signatur, abgelaufenes Token, Access-Token als Refresh-Token) und das Entfernen von `jwt.secret` und `JWT_SECRET` aus den Test-Properties und `docker-compose.yml` .

3. **`65ddb02`** – JaCoCo eingebaut (Plugin 0.8.15, Report bei `./mvnw verify` , `BackendApplication` und `OpenApiConfig` ausgeschlossen) und `lombok.config` , damit Lombok-Code nicht mitgezählt wird. Erster Report: 89 % Zeilen, 79 %

Branches bei 30 Tests. Lücken: `CustomUserDetailsService` (0 %), 404- und 500-Handler, Fehlerpfade der `KeyConfig`.

4. **`640c408`** – Zurück zum globalen Manager von Spring (`config.getAuthenticationManager()`), weil ein zweiter, selbst gebauter Dao-Manager unnötig war. Vorher habe ich die Ursache der Rekursion geklärt: Sie trat nur in einem Testkontext ohne `UserDetailsService` auf (siehe „Neu gelernt"). Belegt habe ich das mit der Log-Zeile „Global AuthenticationManager configured with UserDetailsService bean …" und dem Code von Spring Security 7.0.4. Das Javadoc nennt die Voraussetzung (genau ein `UserDetailsService`) und die Falle, die sonst entsteht.
5. **`ae03d63`** – Testlücken geschlossen:
   - `MeControllerTest`: Tests mit echten Tokens, damit der `JwtAuthenticationConverter` (Claim `roles` → `ROLE_ADMIN`) wirklich durchlaufen wird; 404 für einen gelöschten User
   - `AuthControllerTest`: 500 über den Catch-all-Handler, einzelner Feldfehler beim Sign-up, leeres Refresh-Cookie
   - `AuthServiceTest`: User verschwindet nach dem Passwort-Check
   - neu: `KeyConfigTest` (PEM-Parsing beider Keys und alle Fehlerpfade) und `CustomUserDetailsServiceTest`

   Ergebnis: 46 Tests, 100 % Zeilen, 92,9 % Branches.
6. **`b3ad933`** – Doku auf RSA-Keys umgestellt (`RUNNING.md`, `SECURITY.md`, `CUSTOMIZING.md`): `JWT_PRIVATE_KEY_LOCATION`, `JWT_PUBLIC_KEY_LOCATION` und die `openssl`-Befehle zum Erzeugen des Schlüsselpaars.
7. **`915d18e`** – Ungenutzten Import aus `CustomUserDetailsService` entfernt.
8. **`119a0e9`** – Tests in die Pakete `unit/`, `slice/` und `integration/` verschoben.

Entscheidungen

Entscheidung	Verworfene Alternativen	Begründung
Globaler `AuthenticationManager` von Spring (`config.getAuthenticationManager()`)	eigener `ProviderManager` mit `DaoAuthenticationProvider`	Spring baut aus dem einzigen `UserDetailsService` ohnehin einen Dao-Manager. Ein zweiter wäre doppelt und schwerer zu verstehen. Das Risiko (Rekursion ohne oder mit mehreren `UserDetailsService`) steht im Javadoc.
Zwei Arten von Controller-Tests: `authenticatedAs(...)` und echte Tokens	nur echte Tokens; nur `authenticatedAs(...)`	`authenticatedAs` prüft Controller und `@PreAuthorize` isoliert. Wenige Tests mit echten Tokens prüfen Decoder und Converter. So zeigt ein roter Test genauer, wo der Fehler liegt.
JaCoCo mit Ausschlüssen und `lombok.config`	alle Klassen zählen	`main()` und eine reine Annotations-Klasse sind nicht sinnvoll testbar. Lombok-Getter würden die Zahlen künstlich drücken.
Kein eigener `GlobalExceptionHandlerTest`	eigene Testklasse für den Handler	Der Handler ist ein `@RestControllerAdvice` und läuft in jedem `@WebMvcTest` mit. Die Controller-Tests prüfen ihn genau so, wie er im Betrieb arbeitet.
Fehler über `MockMvc` auslösen und Status plus JSON prüfen	`assertThrows` auf dem Mock	`assertThrows` direkt auf dem Mock prüft nur Mockito, nicht den Handler.
`KeyConfig` mit `ByteArrayResource` testen	Testdateien im Repository	Kein Key-Material im Repo, kein Dateisystem nötig.
Testordner `unit/`, `slice/`, `integration/`	Ordnung nur nach Fachpaket	Man sieht sofort, welche Art Test es ist, und kann jede Art einzeln starten.

Integrationstest erzeugt seine Keys selbst (in Arbeit)	Keys in der CI per `openssl` erzeugen	Ein Test soll nicht von Dateien auf der Maschine abhängen. Sonst scheitert er bei jedem Kollegen ohne Keys.

Neu gelernt

- **AuthenticationProvider**: Ein Prüfer für genau eine Art von Login. Er sagt mit `supports()`, ob er zuständig ist, und prüft mit `authenticate()`. Beispiele: `DaoAuthenticationProvider` (E-Mail und Passwort) und `JwtAuthenticationProvider` (Bearer-Token).
- **AuthenticationManager und ProviderManager**: `AuthenticationManager` ist ein Interface. `ProviderManager` implementiert es mit einer Liste von Providern und einer Schleife: Er fragt jeden Provider, ob er zuständig ist, und gibt den Login an den ersten passenden weiter.
- **Lokaler und globaler Manager**: Jede Filter Chain bekommt einen eigenen, lokalen Manager (bei mir mit dem `JwtAuthenticationProvider`). Darüber gibt es einen globalen Manager als Parent, den Spring aus dem einzigen `UserDetailsService` selbst baut. Scheitert der lokale, fragt er den Parent.
- **Ursache der Rekursion**: Gibt es keinen `UserDetailsService`, kann Spring keinen globalen Manager bauen und gibt einen Platzhalter zurück, der an die `AuthenticationManager`-Bean weiterleitet. War diese Bean selbst `config.getAuthenticationManager()`, zeigte der Platzhalter auf sich selbst. Im alten `MeControllerTest` fehlte der `CustomUserDetailsService`-Import, deshalb trat der Fehler nur dort auf.
- **`@WebMvcTest` und `@Import`**: Ein Slice-Test lädt nur die Web-Schicht (Controller, `@ControllerAdvice`, Filter). `@Configuration`-Klassen und Services muss man mit `@Import` dazuholen.
- **Profile**: `application.properties` gilt immer. `@ActiveProfiles("test")` lädt zusätzlich `application-test.properties`, deren Werte gleichnamige Werte überschreiben. Der Ordner `src/test/resources` entscheidet nur, ob die Datei im Klassenpfad liegt, nicht, ob sie geladen wird.
- **Unit, Slice, Integration, E2E**: Unit testet eine Klasse ohne Spring. Slice testet einen Ausschnitt mit Spring. Integration startet die ganze App (`@SpringBootTest`, H2). E2E steuert die laufenden Systeme von außen (bei mir Playwright mit Postgres).
- **`@Mock` und `@InjectMocks`**: `@InjectMocks` ist kein Mock, sondern das echte Objekt, das getestet wird. Mockito setzt die `@Mock`-Felder in seinen Konstruktor ein.
- **Lambdas und funktionale Interfaces**: `assertThrows` bekommt ein `Executable`, ein Interface mit genau einer Methode. Das Lambda `() -> ...` ist deren Körper. So führt `assertThrows` den Code selbst in einem `try/catch` aus, statt dass er vorher läuft.
- **Statisch und Objekt**: Ein `static`-Feld gibt es einmal pro Klasse, ein normales Feld einmal pro Objekt. JUnit erzeugt für jede Testmethode ein neues Objekt der Testklasse, darum ist `TestKeys.PAIR` `static` und wird nur einmal pro JVM erzeugt.
- **Coverage**: JaCoCo zählt, welche Zeilen und Branches ausgeführt wurden, nicht, ob etwas geprüft wurde. Rot heißt sicher ungetestet, grün heißt nur durchlaufen, gelb heißt nur ein Teil der Branches. Die Zahl findet Lücken, beweist aber keine Qualität.
- **Exceptions und `@RestControllerAdvice`**: Eine unchecked Exception wandert bis zum `DispatcherServlet`. Spring wählt den `@ExceptionHandler` mit dem spezifischsten Typ, die Reihenfolge in der Datei ist egal. Fehler in der Security-Filterkette erreichen den Handler nicht.
- **Deklarierter und echter Typ**: Der Compiler kennt nur den deklarierten Typ (`UserDetails`). Für `getEmail()` braucht es einen Cast oder `assertInstanceOf`, das den gecasteten Wert zurückgibt.

Tests & Verifikation

Test	Ergebnis
`./mvnw test -Dtest=MeControllerTest` nach Umstellung auf den globalen Manager	grün, Log: „Global AuthenticationManager configured with UserDetailsService bean …“
`./mvnw clean verify` nach dem JaCoCo-Einbau	30 Tests grün, 89 % Zeilen, 79 % Branches
`./mvnw clean verify` nach dem Schließen der Lücken	46 Tests grün, 100 % Zeilen (227/227), 92,9 % Branches (26/28)
`./mvnw clean verify` nach dem Verschieben in `unit/`, `slice/`, `integration/`	46 Tests grün, 100 % Zeilen

Nur Unit-Tests: `./mvnw test -Dtest='com/authkit/backend/unit/**/*Test'`	25 Tests grün
Nur Slice-Tests: `./mvnw test -Dtest='com/authkit/backend/slice/**/*Test'`	20 Tests grün
Gegentest Converter (Zeile `setAuthoritiesClaimName("roles")` auskommentieren)	empfohlen, Ausführung nicht erfasst

Die zwei offenen Branches sind ein Filter in der CORS-Konfiguration und der Fallback `error != null` in `ErrorResponse`. Beide habe ich bewusst nicht getestet.

Probleme & Lösungen

- **Problem:** `StackOverflowError` statt 401, wenn ein Refresh-Token als Access-Token geschickt wird. → **Lösung:** Ursache gefunden (Platzhalter zeigt auf sich selbst, nur ohne `UserDetailsService` im Kontext). Erst eigener `ProviderManager`, später wieder der globale Manager mit `CustomUserDetailsService` in jedem Slice-Test.
- **Problem:** Ich bin zuerst davon ausgegangen, dass Spring die `AuthenticationManager`-Bean als globalen Manager nimmt. → **Lösung:** Mit der Log-Zeile beim Teststart und dem Spring-Security-Code widerlegt. Die Bean ist nur der Notnagel.
- **Problem:** Die Admin-Tests waren grün, prüften den `JwtAuthenticationConverter` aber gar nicht. → **Lösung:** zusätzliche Tests mit echten Tokens.
- **Problem:** Der erste 500-Test rief `authService.authenticate(...)` direkt auf dem Mock auf und erreichte den Handler nie. → **Lösung:** Request über `MockMvc` schicken und Status und JSON prüfen.
- **Problem:** Im `CustomUserDetailsServiceTest` verglich eine Zeile `getAuthorities()` mit sich selbst, und das Klassen-Javadoc war aus einem anderen Test kopiert. → **Lösung:** gegen die erwartete Authority `ROLE_USER` prüfen, Javadoc neu.
- **Problem:** Der Kontext-Test braucht lokale Key-Dateien und würde auf jeder fremden Maschine scheitern. → **Lösung:** angefangen, die Keys im Test selbst zu erzeugen (noch offen).

Offene Punkte / Nächste Schritte

- **Integrationstest fertigstellen:** Die halbfertige Arbeit liegt im Stash „WIP integration test: TestKeys.pem/writePemFiles + BackendApplicationTests keys“. Plan: `@TempDir` (statisch) und `@DynamicPropertySource`, die mit `TestKeys.writePemFiles(...)` die Keys schreibt und `jwt.private-key-location` und `jwt.public-key-location` setzt. Danach ein Ablauf Sign-up → Sign-in → `/me` → `/auth/access-tk`.
- **Catch-all-Handler verbessern:** Er gibt `ex.getMessage()` an den Client weiter und loggt nichts. Besser: fester Text in der Antwort, echte Meldung ins Log.
- Optional: Mutation Testing (zum Beispiel PIT), um die Qualität der Tests zu messen statt nur die Abdeckung.

Commit-Verlauf

Datum	Hash	Nachricht
2026-10-05 22:37	`1c89034`	test(auth): cover RS256 token claims, the JWKS endpoint and secured /me routes
2026-10-06 00:42	`e9f21bf`	fix(auth): stop the AuthenticationManager recursion on rejected tokens; extend tests
2026-10-07 22:49	`65ddb02`	build(test): add JaCoCo coverage report
2026-10-07 22:49	`640c408`	refactor(auth): expose Spring's global AuthenticationManager again
2026-10-07 22:49	`ae03d63`	test: close the unit-test gaps found by JaCoCo
2026-10-07 22:49	`b3ad933`	docs: replace JWT_SECRET with the RSA key configuration
2026-10-07 23:40	`915d18e`	chore(auth): remove unused UUID import from CustomUserDetailsService
2026-10-07 23:40	`119a0e9`	test: split the tests into unit, slice and integration